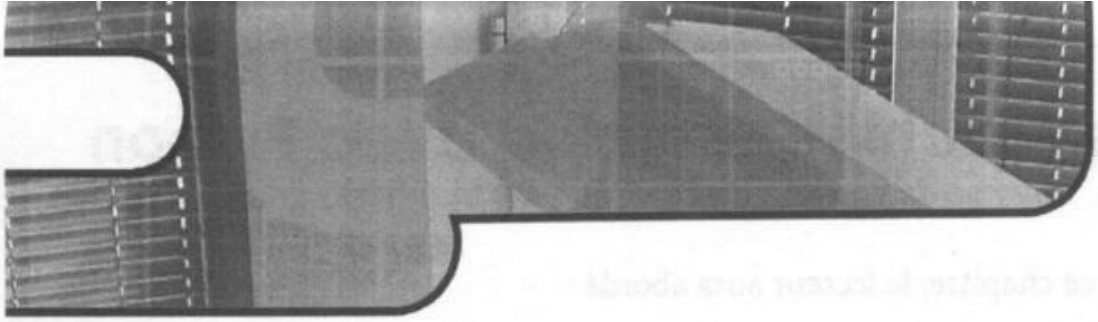


Chapitre 4-4
L'algorithme k-means

1. Objectif du chapitre

Les chapitres précédents ont abordé des exemples de deux types d'algorithmes de Machine Learning : les algorithmes de régression et de classification. Ce chapitre porte sur l'algorithme k-means, appelé l'algorithme des k-moyennes en français, qui est un algorithme simple à comprendre et qui fait partie des algorithmes de clustering les plus connus et les plus utilisés.

L'algorithme k-means a été introduit par J. McQueen en 1967. C'est un algorithme non supervisé qui permet de répartir un ensemble de n observations en k clusters. L'objectif après l'application de l'algorithme k-means sur un jeu de données est que chaque cluster contienne des observations homogènes et que deux observations de deux clusters différents soient hétérogènes.

Les domaines d'application de l'algorithme k-means sont nombreux. Par exemple, il est très utilisé pour la segmentation des clients à des fins de marketing, ou encore pour l'isolation des motifs dans les images, car justement les images présentent souvent des régions homogènes, notamment en matière d'intensité lumineuse.

De manière générale, le succès de l'algorithme k-means et de ses versions réside dans sa simplicité et sa capacité à traiter des données de grande taille.

À la fin de ce chapitre, le lecteur aura abordé :

Le fonctionnement de k-means via des illustrations.

- Les étapes principales de l'algorithme k-means classique.
- L'application de l'algorithme k-means avec Scikit-learn.
- L'impact des valeurs extrêmes sur les performances de l'algorithme k-means.
- La recherche de la valeur optimale du paramètre *K* de l'algorithme k-means.
- Les avantages et les inconvénients ainsi que les variantes de l'algorithme k-means.

2. k-means du point de vue géométrique

Comme précisé au début de ce chapitre, l'algorithme k-means est très intuitif et simple à comprendre. Avant d'entrer dans les détails, il faut noter que k-means, comme tous les algorithmes de clustering, ne nécessite pas l'étiquetage des données, car c'est une procédure non supervisée.

De façon informelle, étant donné n observations à répartir sur $k$clusters, k-means choisit initialement, de manière aléatoire, k observations parmi les n observations, comme étant les centres des k clusters recherchés. Chacune des n observations sera associée au cluster dont le centre est le plus proche parmi les k centres choisis initialement. Une fois que toutes les observations sont associées à leurs clusters respectifs, le centre de chaque cluster est recalculé en fonction des observations qu'il contient. Puis, de nouveau, chacune des observations est associée au cluster dont le centre est le plus proche de cette observation par rapport à tous les centres des autres clusters. Ces opérations de recalcul des centres des clusters puis d'association des observations aux clusters les plus proches sont répétées jusqu'à ce qu'un critère d'arrêt soit atteint.

L'algorithme k-means utilise une fonction pour calculer les distances entre les observations et les centres des clusters. Ce calcul des distances peut être basé sur la distance euclidienne, la distance de Manhattan ou toute autre fonction permettant de mesurer la dissimilarité entre les observations.

© Editions ENI - All rights reserved

Pour mieux comprendre cet algorithme de clustering, cette section déroule l'algorithme k-means sur un exemple simple. Soit six observations a, b, c, d, e et f à répartir sur deux clusters C_1 et C_2 ; supposons que la distance utilisée est la distance euclidienne classique. Ces six observations sont définies dans un espace à deux dimensions et leurs coordonnées sont indiquées dans le tableau suivant :

Axes	a	b	c	d	e	f
x	2	4	2	4	10	10
y	4	4	2	2	2	4

Figure 10-1 : un simple jeu de données avec leurs coordonnées en deux dimensions

Remarque

Pour rappel, la distance euclidienne entre deux observations A=(xA,yA) et B=(xB,yB) est calculée grâce à la formule :

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Avant de faire un traitement quelconque sur les données, il est toujours intéressant de les visualiser lorsque c'est possible. Dans cet exemple, les données sont définies dans un espace à deux dimensions, donc elles peuvent être facilement visualisées sur deux axes comme dans la figure 10-2 ci-dessous.

Remarque

Même lorsque les données sont définies dans un espace à grande dimension, supérieur à deux ou à trois dimensions, il existe des méthodes qui permettent de les visualiser en deux ou trois dimensions, avec une perte d'informations qu'on espère minime. Ces méthodes sont appelées les méthodes de réduction de domaines. Le chapitre Analyse en composantes principales présente l'analyse du même nom, qui est l'une des méthodes de réduction de domaines les plus connues et qui permet d'avoir une vue en deux dimensions des données définies initialement dans un espace à grande dimension.

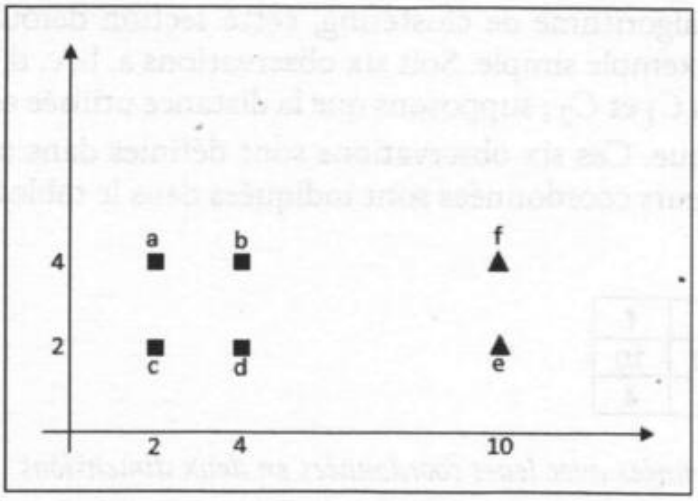


Figure 10-2 : représentation graphique en deux dimensions des données

Ce graphique montre clairement que les observations a, b, c et d, représentées par des carrés, sont très proches entre elles au sens de la distance euclidienne, par rapport aux deux observations e et f. Également, les deux dernières observations, représentées par des triangles, sont très proches entre elles.

Pour cet exemple, en se basant donc sur la distance euclidienne, un algorithme de clustering efficace proposerait certainement de répartir ces six observations dans les deux clusters C_1 et C_2 comme dans la figure 10-3 ci-dessous.

En effet, avec la distance euclidienne, cette répartition est optimale. La section suivante définit de façon plus formelle la notion de solution optimale pour un algorithme k-means et pour un nombre de clusters fixe.

© Editions ENI - All rights reserved

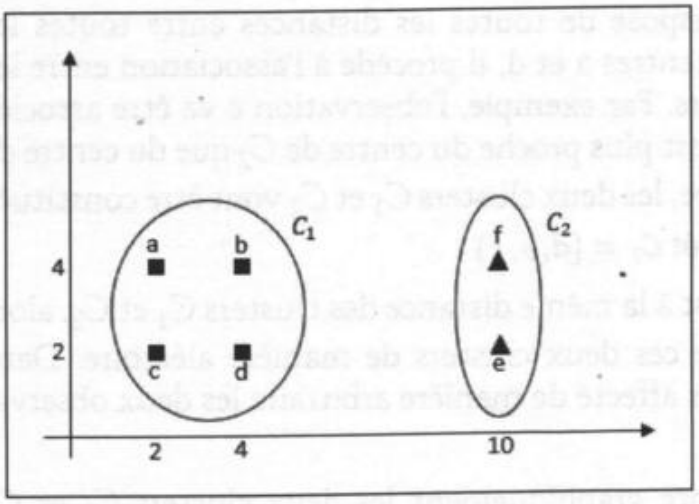


Figure 10-3 : répartition des six points dans les deux classes C_1 et C_2

En suivant les étapes classiques de l'algorithme k-means, le résultat optimal de la figure 10-3 peut être obtenu comme suit :

1. L'algorithme k-means commence initialement par sélectionner de façon aléatoire deux observations parmi les six observations disponibles. Les deux observations ainsi sélectionnées vont être considérées comme les centres des deux clusters recherchés C_1 et C_2. Ici, nous supposons que l'algorithme k-means recherche un nombre de clusters qui est égal à 2.

   Dans cet exemple, supposons que les deux observations a et d sont sélectionnées aléatoirement. Ces deux observations vont être considérées comme les centres respectifs des clusters C_1 et C_2. L'algorithme k-means calcule les distances entre chacune des six observations avec les centres a et d. Les résultats sont reportés dans le tableau ci-dessous :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1=a=(2,4)	0	2	2	2.8284	8.2462	8
Centre de C_2=d=(4,2)	2.8284	2	2	0	6	6.3245

Figure 10-4 : distances entre les observations et les centres de C_1 et C_2

2. Une fois que k-means dispose de toutes les distances entre toutes les observations et les deux centres a et d, il procède à l'association entre les observations et les clusters. Par exemple, l'observation e va être associée au cluster C_2, puisqu'elle est plus proche du centre de C_2 que du centre de C_1. À la suite de cette étape, les deux clusters C_1 et C_2 vont être constitués comme suit $C_1 = \{a, b, c\}$ et $C_2 = \{d, e, f\}$.

   Lorsqu'une observation est à la même distance des clusters C_1 et C_2, alors elle est affectée à l'un de ces deux clusters de manière aléatoire. Dans notre exemple nous avons affecté de manière arbitraire les deux observations b et c au cluster C_1.

   La figure suivante présente graphiquement les deux clusters C_1 et C_2 obtenus à la suite de cette étape :

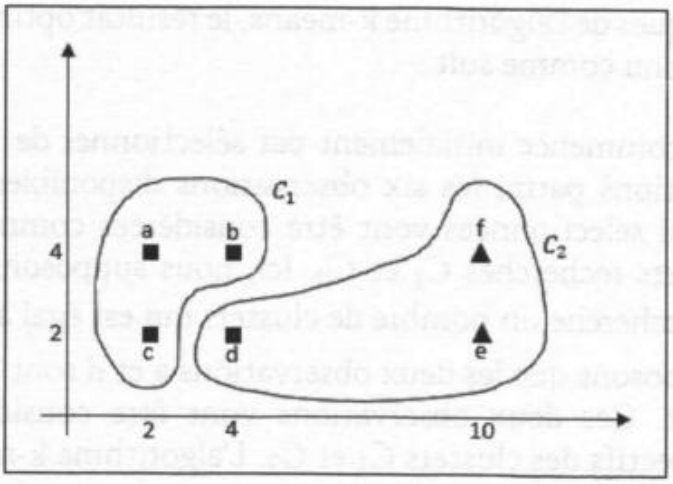


Figure 10-5 : affectation des observations aux clusters C_1 et C_2 après la première itération

3. Lors de la première étape, l'algorithme k-means a choisi de façon aléatoire a et d en tant que centres des deux clusters recherchés. À l'étape 2, les deux clusters C_1 et C_2 ont été concrètement construits.

© Editions ENI - All rights reserved

À cette étape, l'algorithme k-means calcule le centre du cluster C_1 en utilisant les observations a, b et c et calcule le centre du cluster C_2 en utilisant les observations d, e et f.

Ainsi : le centre de $C_1 = \left(\frac{1}{3}(2+2+4), \frac{1}{3}(4+4+2)\right) = (2.66, 3.33)$

le centre de $C_2 = \left(\frac{1}{3}(4+10+10), \frac{1}{3}(2+2+4)\right) = (8, 2.66)$

Les centres des deux clusters C_1 et C_2 sont représentés avec le symbole étoile dans la figure 10-6 ci-dessous :

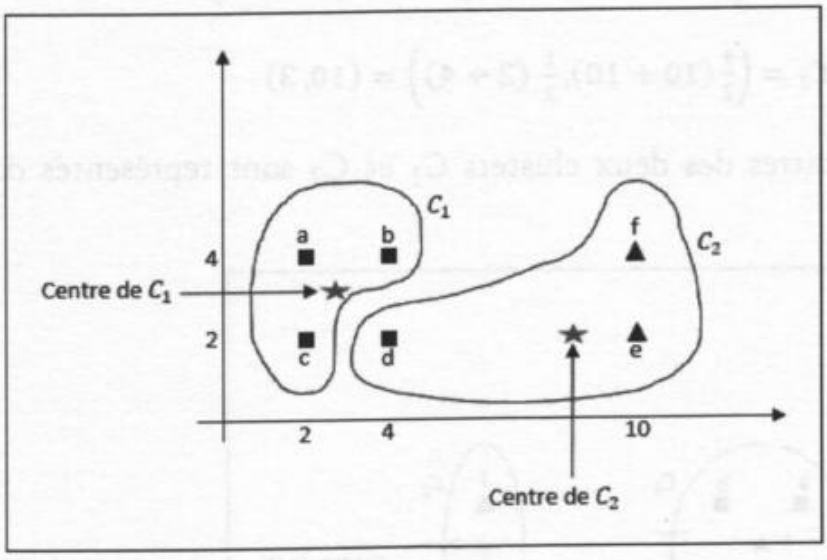


Figure 10-6 : les centres des clusters C_1 et C_2

4. À cette étape, les six observations sont réparties de nouveau sur les deux clusters C_1 et C_2 en se basant sur les nouveaux centres calculés à l'étape précédente. Les distances entre les observations et les centres de C_1 et C_2 sont reportées dans le tableau suivant :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1= (2.66,3.33)	0.9404	1.4981	1.4847	1.8879	7.4595	7.3505
Centre de C_2= (8,2.66)	6.1478	4.2184	6.0361	4.0540	2.1060	2.4074

Figure 10-7 : distances entre les observations et les centres de C_1 et C_2

5. Après la dernière répartition, les deux clusters deviennent :

   $C_1 = \{a, b, c, d\}$ et $C_2 = \{e, f\}$

Remarque

Remarquez que les deux clusters C_1 et C_2 construits à cette étape sont différents des deux clusters construits à l'étape 2:

6. Avec les deux nouveaux clusters C_1 et C_2, les centres deviennent :

   Le centre de $C_1 = \left(\frac{1}{4}(2+4+2+4), \frac{1}{4}(4+4+2+2)\right) = (3,3)$

   Le centre de $C_2 = \left(\frac{1}{2}(10+10), \frac{1}{2}(2+4)\right) = (10,3)$

Les nouveaux centres des deux clusters C_1 et C_2 sont représentés dans la figure suivante :

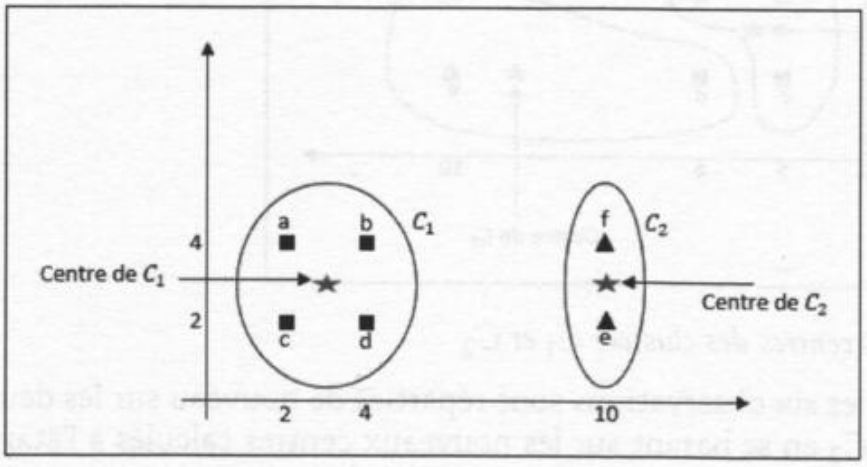


Figure 10-8 : les centres des clusters C_1 et C_2

© Editions ENI - All rights reserved

7. De nouveau, les distances entre les observations et les nouveaux centres des deux clusters C_1 et C_2 sont calculées. Le tableau suivant donne les distances des six observations avec les nouveaux centres calculés à l'étape précédente :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1= (3,3)	1.4142	1.4142	1.4142	1.4142	7.0710	7.0710
Centre de C_2= (10,3)	8.0622	6.0827	8.0622	6.0827	1	1

Figure 10-9 : distances entre les observations et les centres des clusters C_1 et C_2

8. Avec les distances du tableau précédent, les six observations sont réparties sur les deux clusters C_1 et C_2 comme suit $C_1 = \{a, b, c, d\}$ et $C_2 = \{e, f\}$.

Ces derniers clusters C_1 et C_2 sont exactement les mêmes que les clusters obtenus à l'étape 5. Par conséquent, l'algorithme s'arrête puisque, s'il recalcule les centres de ces nouveaux clusters, il trouvera les mêmes résultats que ceux obtenus à l'étape 6.

Les deux clusters C_1 et C_2 obtenus constituent la solution optimale de cet exemple. Il faut noter que si les centres initiaux choisis à l'étape 1 avaient été par exemple $\{b, d\}$ et non pas $\{a, d\}$, les résultats auraient été très différents. Ce dernier constat sera confirmé dans la suite de ce chapitre, lorsque l'algorithme k-means sera appliqué avec Scikit-learn.

3. k-means du point de vue algorithmique

L'exemple de la section précédente montre le mécanisme de base de l'algorithme k-means. Les deux paramètres principaux de cet algorithme sont le nombre de clusters k à construire et la fonction utilisée pour le calcul des distances entre les observations. Le schéma général de l'algorithme k-means est présenté ci-dessous :

```
Entrées : un ensemble d'observations X = {X1, X2, ..., Xi, ..., Xn}
          un nombre entier K. correspondant au nombre de
          clusters à construire
          une fonction de distance 𝔽
```

```
Sortie : une partition 𝒫 de l'ensemble X en K clusters
1- Initialisation aléatoire des centres C̄ᵢ, 1 ≤ i ≤ K
2- Répété :
    a) Pour chaque observation Xᵢ, calculer les distances 𝔽(Xᵢ, C̄ⱼ)
       pour tous les j=1..K
    b) Association de chaque observation Xᵢ au cluster
       dont le centre est le plus proche de Xᵢ
    c) Recalculer les centres C̄ᵢ, 1 ≤ i ≤ K
3- Reprendre au point (2) jusqu'à ce que les centres C̄ᵢ, 1 ≤ i ≤ K
deviennent stables
```

L'algorithme k-means prend en entrée un ensemble d'observations $\{X_1, X_2, \ldots, X_i, \ldots, X_n\}$, un paramètre k qui correspond au nombre de clusters que l'algorithme doit construire et une fonction de distance $\mathbb{F}$.

La première étape consiste en l'initialisation aléatoire des centres des k clusters, c'est-à-dire que k-means tire aléatoirement k observations qui seront considérées comme les k centres des k clusters recherchés. Puis il réalise un ensemble d'itérations jusqu'à ce qu'un critère d'arrêt soit atteint.

À chaque itération, en utilisant la fonction de distance choisie, k-means calcule la distance entre chaque observation X_i et les k centres. Pour la première itération, ces k centres sont ceux tirés aléatoirement à la première étape. Chaque X_i sera associé au cluster dont le centre est le plus proche de X_i. À ce stade, k-means aura construit une première partition en k clusters des observations $\{X_1, X_2, \ldots, X_i, \ldots, X_n\}$. L'algorithme k-means s'arrête lorsque les k clusters deviennent stables, c'est-à-dire lorsque l'algorithme obtient des clusters identiques entre deux itérations.

Comme montré dans l'exemple précédent, les résultats de l'algorithme k-means sont fortement influencés par le choix des k centres initiaux. Pour aboutir à une répartition efficace, il faudrait exécuter l'algorithme k-means plusieurs fois et sélectionner la partition la plus performante.

© Editions ENI - All rights reserved

Pour une valeur de k fixe, c'est-à-dire pour un nombre de clusters fixe, la meilleure partition est celle qui minimise la quantité suivante et qui correspond à l'inertie totale associée à l'ensemble des clusters $\sum_{i=1}^{K} \sum_{x \in C_i} d^2(x, \overline{C}_i)$

Avec $\overline{C}_i$ le centre du cluster C_i et $d^2(x, \overline{C}_i)$ le carré de la distance séparant le point x du centre de son cluster C_i.

Cette quantité correspond aux carrés des distances entre les observations et les centres des clusters auxquels elles appartiennent. À chaque itération de l'algorithme k-means, cette quantité s'approche de son minimum.

À la fin de son exécution, l'algorithme k-means ne garantit pas l'optimalité de la partition trouvée. En effet, la convergence de l'algorithme k-means est fortement influencée par le choix des centres initiaux, et souvent il converge vers des optimums locaux.

4. Application de k-means avec Scikit-learn

L'application de l'algorithme k-means avec Scikit-learn est très simple et intuitive ! Les exemples ci-dessous montrent comment appliquer cet algorithme sur des jeux de données afin de regrouper les données dans des clusters.

Notez dès à présent que dans les exemples de cette section le nombre de clusters à construire, c'est-à-dire la valeur du paramètre k, est connu à l'avance. Dans la suite de ce chapitre, nous allons voir comment tenter de trouver la valeur optimale de ce paramètre k.

Commençons avec un premier exemple dans lequel nous utilisons les mêmes données que dans l'illustration de la figure 10-2 précédente. Pour réaliser cet exemple, suivez les étapes ci-après.

- Créez un nouveau notebook Jupyter, puis saisissez et exécutez les instructions suivantes dans une nouvelle cellule ; vous pouvez également utiliser le notebook *k-means-01.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
%matplotlib inline
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt

data = pd.read_csv('../Data/exemple_simple_kmeans-1.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,5)
```

Ces instructions vont tout simplement permettre de charger les modules nécessaires à notre exemple et de lire les données du fichier *exemple_simple_k-means-1.csv*. Ces données seront affichées dans un graphique. Remarquez l'import du module `KMeans` qui sera utilisé pour instancier et utiliser l'algorithme k-means.

L'exécution de ce code affiche un graphique comme celui de la figure ci-dessous :

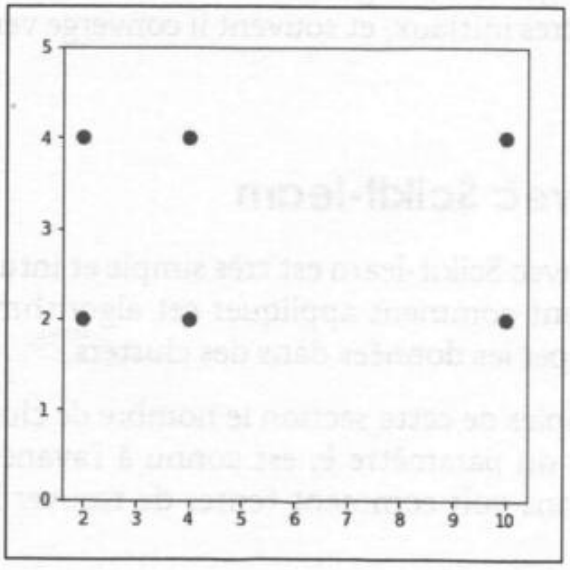


Figure 10-10 : représentation des données en deux dimensions

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
model = KMeans(n_clusters=2)
model.fit(data)
```

Dans ce dernier code, la variable `model` permet de créer une instance de l'algorithme k-means. Ici, nous l'avons initialisée avec le paramètre `n_clusters=2`. Ce paramètre permet d'indiquer le nombre de clusters à construire et qui est égal à 2 pour notre exemple.

© Editions ENI - All rights reserved

Une fois que nous avons une instance de l'algorithme k-means, pour construire des clusters à partir d'un jeu de données, il suffit de lancer la fonction `fit` en lui indiquant le jeu de données dans lequel les clusters seront recherchés.

À la fin de l'exécution de la fonction `fit`, les clusters sont construits.

Notez bien que, contrairement à tous les algorithmes que nous avons abordés lors des chapitres précédents, cette fois nous n'avons pas eu à séparer nos données pour la création d'un jeu de données d'entraînement et d'un jeu de données de test. En effet, ce n'est pas nécessaire avec k-means vu que, pour rappel, nous réalisons un apprentissage non supervisé !

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.labels_
```

L'attribut `labels_` contient un tableau dont le nombre d'éléments est égal au nombre de données analysées avec la fonction `fit`.

L'exécution de cette dernière cellule affiche le résultat suivant :

```
array([0, 0, 0, 0, 1, 1])
```

Ce résultat correspond tout simplement aux libellés associés à nos données. Nous avons six points à répartir dans deux clusters. Les quatre premiers points sont donc associés au cluster `0` et les deux derniers points sont associés au cluster `1`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.cluster_centers_
```

L'attribut `cluster_centers_` stocke les coordonnées des centres des clusters construits.

Cette instruction affiche le résultat suivant :

```
array([[3.,3.],
       [10., 3.]])
```

Ce résultat montre que le centre du premier cluster a comme coordonnées `(3, 3)` et le centre du deuxième cluster est défini par `(10, 3)`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.inertia_
```

L'attribut `inertia_` correspond à l'inertie totale des deux clusters. L'exécution de cette dernière instruction donne le résultat suivant :

```
10.0
```

Cela veut dire que la somme des carrés des distances qui séparent les différents points des centres de leurs clusters respectifs est égale à 10.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes afin de visualiser nos résultats :

```
colormap = np.array(['green','blue'])
plt.figure( figsize=(5, 5))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=20)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,5)
```

Ce code permet d'afficher les données représentées par des points de notre exemple dans un graphique avec l'étiquette ou le chiffre correspondant au cluster associé à chaque point. Également, les centres des deux clusters sont représentés sur ce graphique par des points de couleur rouge.

© Editions ENI - All rights reserved

Ce résultat correspond bien à celui souhaité, comme le montre la figure ci-dessous :

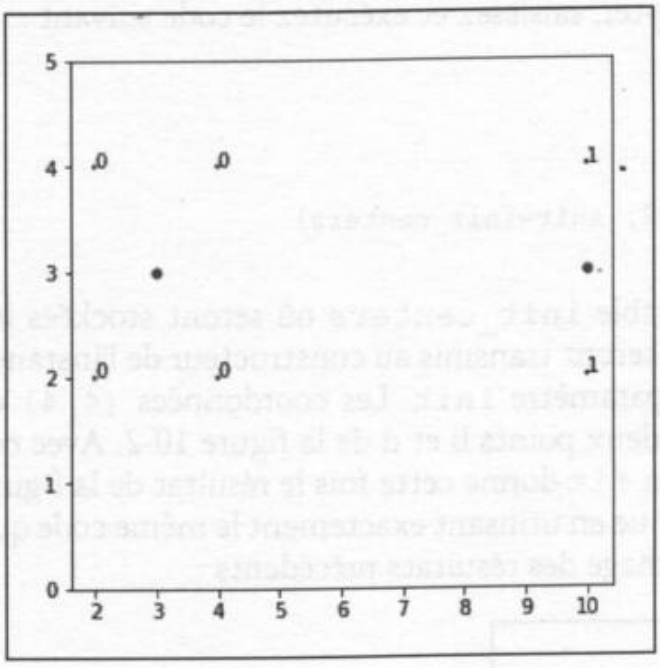


Figure 10-11 : k-means avec k=2 appliqué sur les données exemple_simple_kmeans-1.csv

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin d'appliquer le modèle sur de nouvelles données :

```
print(model.predict([[1,1],[11,1]]))
```

Cette instruction affiche le résultat suivant :

```
[0 1]
```

Dans ces résultats, le cluster prédit par le modèle pour le point défini par les coordonnées `(X=1, Y=1)` est le cluster `0` et le cluster prédit pour le point défini avec les coordonnées `(X=11, Y=1)` est le cluster `1`.

Lorsque k-means a été présenté au début de ce chapitre, nous avons mentionné l'importance du choix des centres des clusters initiaux. Pour illustrer l'impact de cette étape d'initialisation de k-means, nous allons exécuter de nouveau k-means sur les mêmes données que précédemment, mais en lui indiquant de manière explicite les centres des clusters initiaux.

Nous allons choisir les points b et d (voir la figure 10-2) comme étant les centres des deux clusters initiaux.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
init_centers= np.array([
    [4,4],
    [4,2],
])
model = KMeans(n_clusters=2, init=init_centers)
model.fit(data)
```

Ce code permet de créer la variable `init_centers` où seront stockées les coordonnées de deux points qui seront transmis au constructeur de l'instance de l'algorithme k-means via le paramètre `init`. Les coordonnées `(4,4)` et `(4,2)` correspondent bien aux deux points b et d de la figure 10-2. Avec ces deux centres initiaux, la fonction `fit` donne cette fois le résultat de la figure suivante. Cette figure a été obtenue en utilisant exactement le même code que celui qui nous a servi pour l'affichage des résultats précédents :

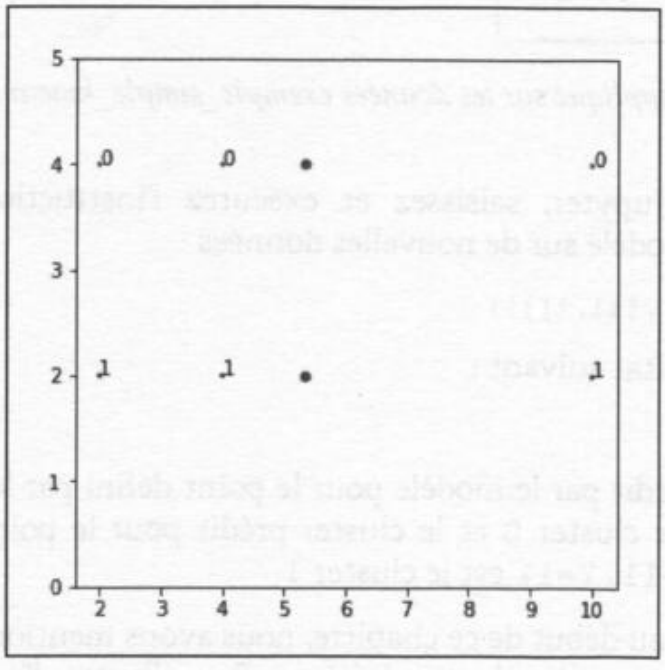


Figure 10-12 : k-means avec k=2 appliqué sur les données exemple_simple_kmeans-1.csv avec les centres initiaux b et d

© Editions ENI - All rights reserved

Cette fois, les observations ont été réparties différemment sur les deux clusters par rapport à l'exécution précédente.

Remarquez que cette nouvelle répartition est de moindre qualité que celle obtenue lors de la première exécution. En effet, dans cette dernière répartition, les trois observations en bas sont regroupées dans le même cluster et les observations en haut sont également regroupées dans un même cluster. Pourtant, au sens de la distance euclidienne, l'observation tout en bas à gauche est beaucoup plus proche de l'observation tout en haut à gauche que de l'observation tout en bas à droite !

Ces derniers résultats montrent concrètement l'importance du tirage initial des centres des clusters.

Lors du premier clustering réalisé, l'attribut était égal à `10`. Voyons quelle est la valeur de l'inertie totale associée au clustering de ce nouveau modèle.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.inertia_
```

Cette fois, le résultat est :

```
69.33333333333334
```

Force est de constater une grande différente entre les deux inerties des clusterings ! Plus les données sont réparties correctement, moindre est la valeur de l'inertie totale.

Voyons maintenant ce que donne la prédiction des clusters des deux points `(X=1,Y=1)` et `(X=11,Y=1)`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de réaliser la prédiction :

```
print(model.predict([[1,1],[11,1]]))
```

Cette instruction affiche le résultat suivant :

```
[1 1]
```

Le point `(X=1, Y=1)` est beaucoup plus proche du point `a=(2, 4)`, mais avec ce nouveau modèle il serait placé dans le même cluster que le point `(X=11, Y=1)` ! Ce qui prouve que ce nouveau modèle est de faible qualité.

5. L'algorithme k-means et les valeurs extrêmes

L'algorithme k-means utilise la notion de distance afin de calculer les centres des clusters recherchés. Bien sûr, la nature de la fonction de calcul de distance utilisée a une influence directe sur les résultats de k-means. Cependant, quelle que soit la fonction de distance utilisée, l'algorithme k-means est sensible aux valeurs extrêmes, comme nous allons le montrer dans l'exemple suivant.

- Créez un nouveau notebook, puis saisissez et exécutez le code suivant ; vous pouvez utiliser le notebook *k-means-02.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
data = pd.read_csv('../Data/exemple_simple_kmeans-2.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,6)
plt.xlim(0,15)
```

© Editions ENI - All rights reserved

Ce bout de code permet tout simplement la lecture et l'affichage dans un graphique des données du fichier *exemple_simple_kmeans-2.csv*. Vous obtenez le graphique suivant :

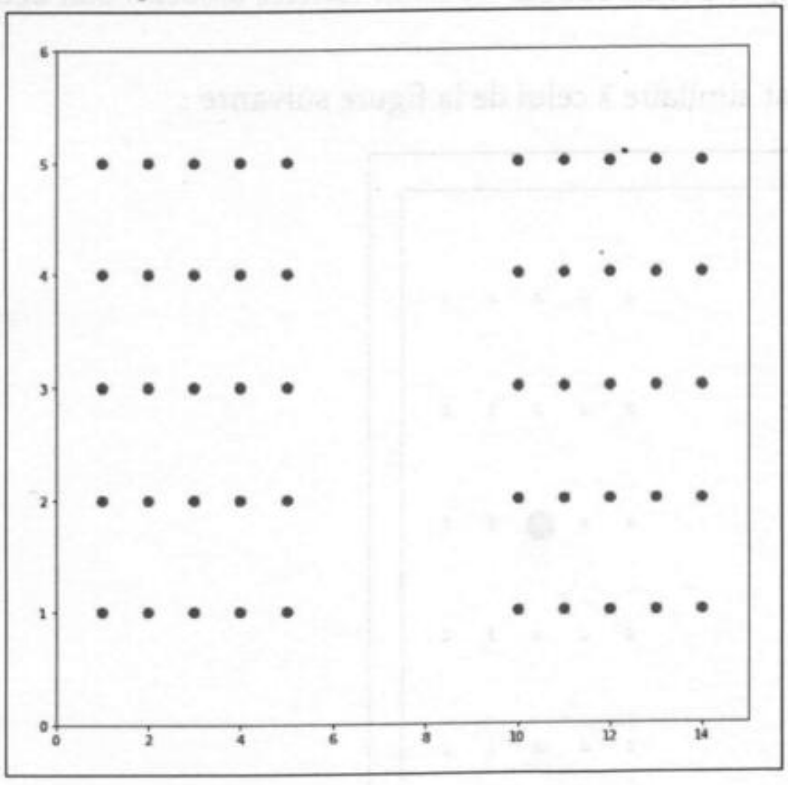


Figure 10-13 : représentation graphique des données définies dans le fichier exemple_simple_kmeans-2.csv

Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
model = KMeans(n_clusters=2)
model.fit(data)

colormap = np.array(['green','blue'])
plt.figure( figsize=(10, 10))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=500)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,6)
plt.xlim(0,15)
```

Les deux premières instructions de ce code appliquent l'algorithme k-means pour rechercher deux clusters. Le reste du code permet d'afficher les résultats obtenus en indiquant pour chaque point le numéro de son cluster. Également, ce code affiche avec de gros points rouges les deux centres associés aux deux clusters trouvés.

Vous obtenez un résultat similaire à celui de la figure suivante :

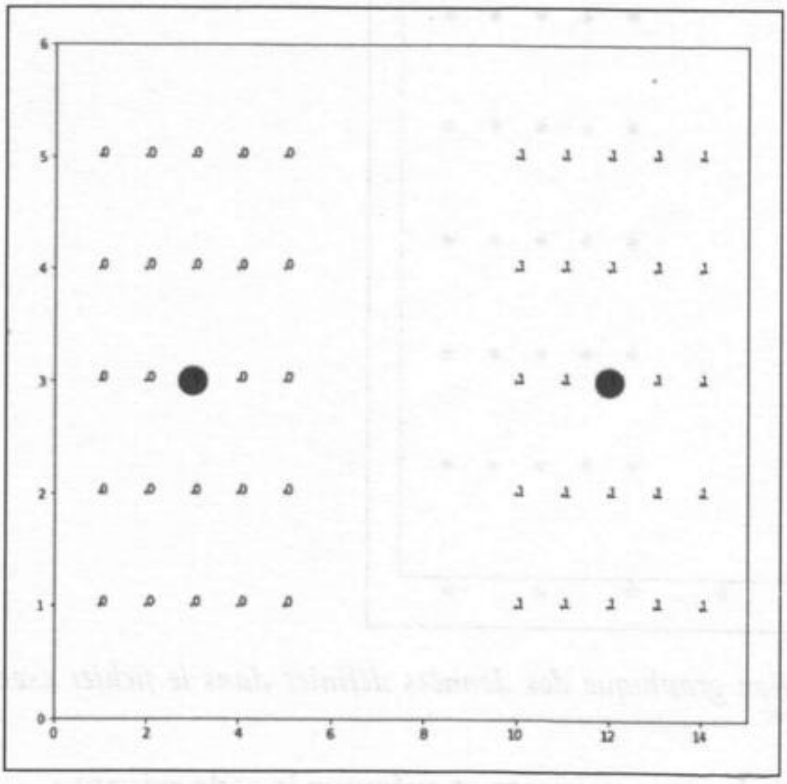


Figure 10-14 : résultat de k-means avec k=2 sur les données du fichier exemple_simple_kmeans-2.csv

Ce résultat correspond au résultat attendu puisqu'en observant le graphique de la figure 10-13, on distingue bien deux clusters.

Dans la suite de cet exemple, nous allons exécuter plus ou moins le même code que celui des deux dernières cellules sur un jeu de données qui est celui utilisé précédemment, auquel nous ajoutons un seul point que nous qualifions de point extrême au vu de ses coordonnées `(x, y)`.

© Editions ENI - All rights reserved

En effet, nous utilisons cette fois les données du fichier *exemple_simple_k-means-2-bis.csv* contenant toutes les données du fichier précédent plus un point avec les coordonnées (X=10, Y=40).

- Dans le même notebook, créez une nouvelle cellule Jupyter puis saisissez et exécutez le code suivant :

```
data = pd.read_csv('../Data/exemple_simple_kmeans-2-bis.csv',
sep=',')
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,45)
```

Ce code lit et affiche dans un graphique les points du fichier *exemple_simple_k-means-2-bis.csv*. L'exécution de cette dernière cellule donne le résultat de la figure suivante :

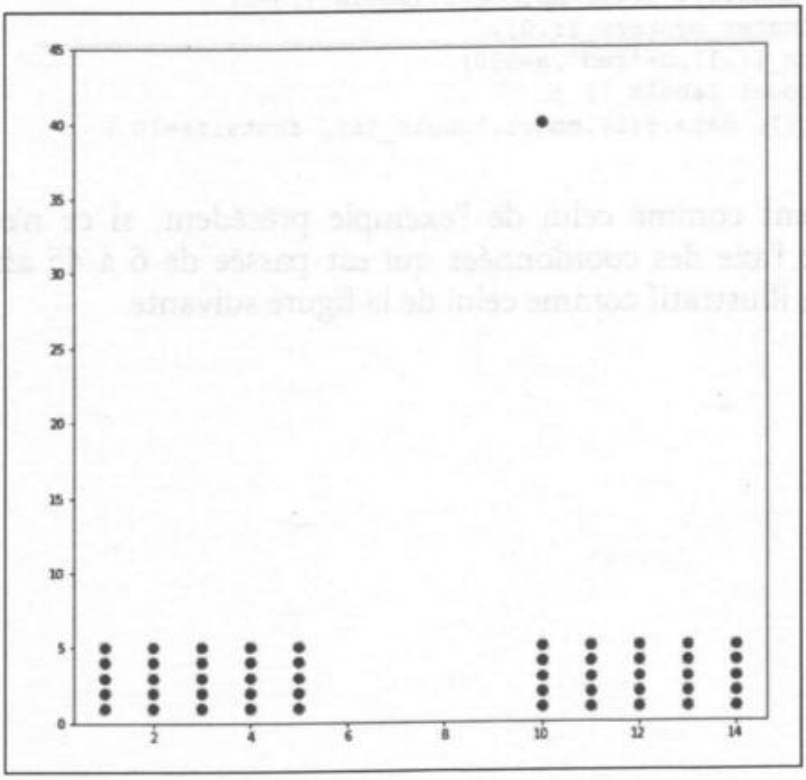


Figure 10-15 : affichage du nuage de points défini dans le fichier exemple_simple_k-means-2-bis.csv

La figure ci-dessus montre clairement que le nouveau point défini avec les coordonnées `(10,40)` est sensiblement loin de tous les autres points et il peut être considéré comme un point extrême. Le reste des points, autres que ce nouveau point, peuvent en toute légitimité former deux clusters différents comme dans l'exemple précédent. On peut se demander comment k-means va se comporter face à de telles situations si on lui demande de créer deux clusters. Va-t-il considérer les deux clusters comme dans l'exemple de la figure 10-14 en ajoutant le nouveau point au cluster à droite? Pour voir concrètement comment k-means réagira, poursuivons avec notre exemple.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
model = KMeans(n_clusters=2)
model.fit(data)
colormap = np.array(['green','blue'])
plt.figure( figsize=(10, 10))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=500)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,45)
```

Ce code est exactement comme celui de l'exemple précédent, si ce n'est l'échelle d'affichage de l'axe des coordonnées qui est passée de 6 à 45 afin d'obtenir un graphique illustratif comme celui de la figure suivante.

© Editions ENI - All rights reserved

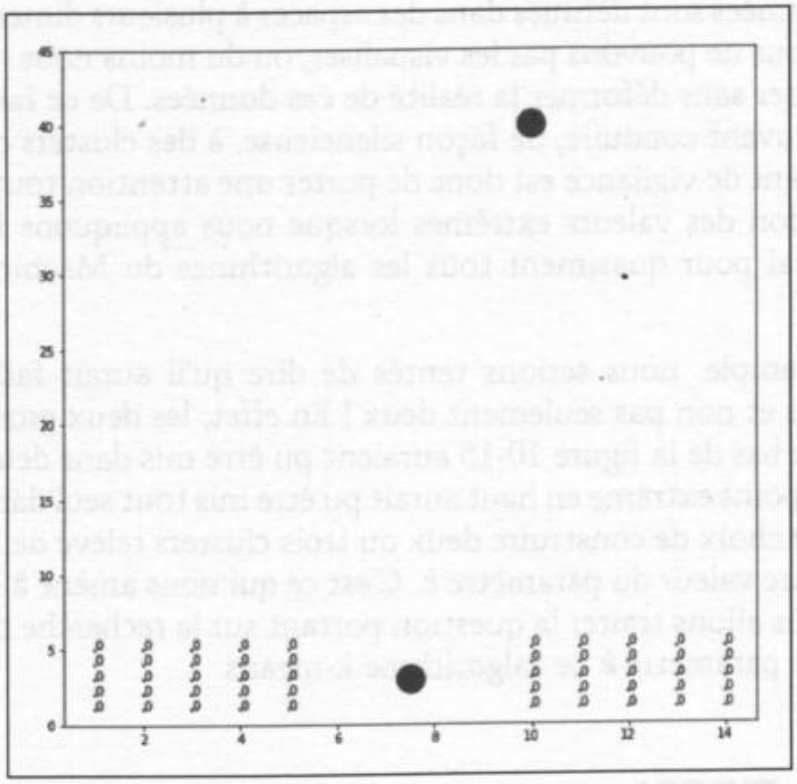


Figure 10-16 : résultat de k-means avec k=2 sur les données du fichier exemple_simple_kmeans-2-bis.csv

Ce résultat montre que le point extrême en haut forme un cluster à lui tout seul ! Et tous les autres points sont positionnés dans un seul et même cluster.

Bien sûr, des clusters comme ces deux derniers ne sont généralement pas souhaités. Il faut noter que dans notre exemple les données sont définies dans un espace à deux dimensions et c'est pour cette raison que nous pouvons facilement les visualiser et reconnaître que les deux clusters trouvés avec k-means dans cet exemple ne sont pas les meilleurs clusters que ne pouvons trouver dans l'absolu. Également, dans notre exemple, nous avons très peu de données pour réaliser des graphiques pédagogiques et montrer l'influence que peuvent avoir les valeurs extrêmes sur le fonctionnement de l'algorithme k-means.

Dans la pratique, les données sont définies dans des espaces à plusieurs dimensions, à tel point que nous ne pouvons pas les visualiser, ou du moins nous ne pouvons pas les visualiser sans déformer la réalité de ces données. De ce fait, les valeurs extrêmes peuvent conduire, de façon silencieuse, à des clusters de moindre qualité. Un point de vigilance est donc de porter une attention toute particulière à la détection des valeurs extrêmes lorsque nous appliquons k-means, et cela reste vrai pour quasiment tous les algorithmes du Machine Learning.

Dans notre dernier exemple, nous serions tentés de dire qu'il aurait fallu rechercher trois clusters et non pas seulement deux ! En effet, les deux groupements de données en bas de la figure 10-15 auraient pu être mis dans deux clusters différents et le point extrême en haut aurait pu être mis tout seul dans un troisième cluster. Le choix de construire deux ou trois clusters relève de la recherche d'une meilleure valeur du paramètre *k*. C'est ce qui nous amène à la section suivante où nous allons traiter la question portant sur la recherche de la valeur optimale de ce paramètre *k* de l'algorithme k-means.

6. Choisir le k de k-means

A priori, lorsque nous sommes face à un grand volume de données sur lequel nous souhaitons appliquer l'algorithme k-means, nous ignorons quel est le nombre de clusters qu'il serait judicieux de construire. On peut imaginer d'exécuter l'algorithme k-means avec plusieurs valeurs du paramètre *k* et de choisir le partitionnement des données le plus pertinent ! Justement, toute la problématique réside dans la définition de ce qu'est un partitionnement pertinent !

Dans la réalité, l'algorithme k-means est appliqué dans l'un des deux contextes suivants.

© Editions ENI - All rights reserved

1. Le premier contexte concerne le cas où nous avons une idée, plus ou moins précise, de ce que nous recherchons dans les données. Par exemple, une entreprise de vente de produits électroménagers, tels que les machines à laver ou les lave-vaisselles, utilise k-means afin de réaliser une segmentation de ses clients dans le but de détecter les profils éventuellement intéressés par certaines options de ses appareils. Ici, k-means peut être orienté afin que le clustering soit réalisé autour des options des produits proposés. Pour cela, il suffit de donner plus de poids aux variables décrivant les options de ces produits afin d'indiquer à k-means que les clients qui achètent plus ou moins les mêmes options soient considérés comme faisant partie des mêmes clusters. Dans cet exemple, une fois que k-means est exécuté plusieurs fois, avec à chaque fois une valeur différente du paramètre *k*, nous pouvons analyser les clusters construits en nous appuyant sur l'idée que les clients d'un même cluster auraient tendance à acheter des produits avec les mêmes options. Si, par exemple, avec une valeur de *k* égale à 3 les clusters retournés par k-means contiennent des profils très divergents par rapport aux options de leurs produits achetés, alors nous pouvons considérer que la valeur 3 pour le paramètre *k* n'est pas suffisante pour construire des clusters intéressants ; il serait donc judicieux d'essayer avec des valeurs supérieures à 3 pour ce paramètre *k*. Dans ce cas, notre analyse pour trouver des valeurs intéressantes du paramètre *k* est basée sur un point de vue purement métier. Bien sûr, cela ne nous garantit en rien que l'augmentation de la valeur du paramètre *k* aboutira à des clusters de meilleure qualité. D'ailleurs, il n'est pas rare que k-means soit utilisé sans succès lorsque nous avons un point de vue métier sur ce que nous recherchons, mais cela a le mérite de nous donner le pouvoir de décider si le clustering réalisé par k-means est intéressant ou pas par rapport à l'objectif.

2. Le deuxième contexte concerne le cas où nous n'avons aucune idée de ce que nous recherchons. Dans ce cas, nous avons un grand volume de données à partir duquel nous allons tenter d'extraire de nouvelles connaissances avec k-means. En pratique, k-means est lancé plusieurs fois, avec des valeurs différentes du paramètre *k* pour chaque exécution, puis nous analysons les clusters construits afin de voir s'il n'y aurait pas une information intéressante à exploiter. Ici nous supposons que pour chaque valeur du paramètre *k*, k-means a trouvé un ou plusieurs facteurs qui l'ont poussé à regrouper les données dans les clusters construits. Donc, pour chaque cluster, nous essayons de comprendre pourquoi k-means a regroupé les données ensemble dans ce cluster et tentons également de savoir pourquoi certaines données sont séparées dans des clusters différents. Cela peut mettre en lumière des informations sur nos données que nous n'avions pas forcément avant la construction de ces clusters !

   Dans ce deuxième contexte d'utilisation de l'algorithme k-means, nous n'avons rien pour nous guider dans notre choix des valeurs du paramètre *k*. Cependant, certaines méthodes peuvent nous aider et nous orienter vers des valeurs intéressantes du paramètre *k*. Justement, les deux prochaines sous-sections expliquent deux de ces méthodes les plus utilisées.

Quel que soit le contexte dans lequel k-means est appliqué, les méthodes d'aide au choix des valeurs du paramètre *k* seront applicables et intéressantes.

Remarque

Il faut noter dès à présent que les deux méthodes présentées ci-dessous sont purement mathématiques et leur utilisation est à conjuguer avec notre analyse qui, elle, n'a de sens que si elle est orientée métier.

© Editions ENI - All rights reserved

6.1 Déterminer k avec la méthode Elbow

La méthode Elbow (méthode du coude en français) est parmi les méthodes les plus couramment utilisées afin d'indiquer la qualité d'un clustering. Elle est simple et intuitive, car basée uniquement sur le calcul de l'inertie totale d'un clustering complet.

Pour trouver le nombre optimal de clusters à construire avec l'algorithme k-means, la méthode Elbow propose d'exécuter plusieurs fois k-means en variant le paramètre *k* dans un intervalle de valeurs, par exemple `k=[1,2,...10]`, puis de calculer l'inertie totale de chaque clustering et afficher la courbe associée aux variations de cette inertie par rapport aux variations de la valeur du paramètre *k*. Avec la courbe ainsi construite, nous espérons trouver la valeur optimale du paramètre *k*.

■Remarque

Pour rappel, la formule de calcul de l'inertie d'une partition est donnée précédemment dans ce chapitre à la section k-means du point de vue algorithmique.

La méthode Elbow en pratique

Le code suivant donne un exemple d'utilisation de la méthode Elbow pour déterminer le nombre de clusters à considérer avec k-means.

▶Créez un nouveau notebook, puis saisissez et exécutez le code suivant ; vous pouvez utiliser le notebook *k-means-03.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
%matplotlib inline
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
data=pd.read_csv('../Data/exemple_simple_kmeans-3.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
```

Ce code permet d'importer les modules nécessaires à notre exemple et d'afficher dans un graphique les données du fichier *exemple_simple_kmeans-3.csv* (voir la figure suivante). Ces données sont définies par deux variables `X` et `Y`.

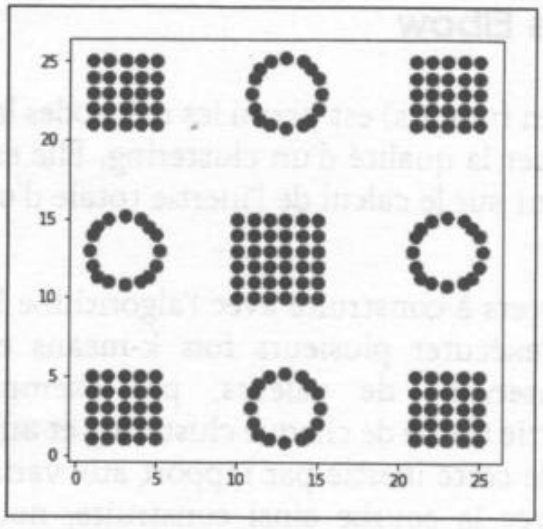


Figure 10-17 : représentation graphique des données du fichier exemple_simple_k-means-3.csv

Cette figure montre clairement que ces données sont organisées en neuf groupes. Pour ce jeu de données, nous pouvons affirmer sans hésiter qu'un clustering correct serait donc organisé en neuf clusters !

Bien sûr, ici comme dans l'exemple précédent, insistons sur le fait que les données que nous avons utilisées sont définies dans un espace à deux dimensions, nous permettant de les visualiser facilement et de savoir qu'un clustering avec neuf clusters serait le plus intéressant. Cependant, dans la réalité, la visualisation des données dans un graphique à deux dimensions n'est pas aussi facile, voire impossible. Ici, nous avons choisi d'utiliser des données simples afin d'illustrer le fonctionnement de l'algorithme k-means.

Pour jouer le jeu, supposons que nous n'avons pas vu le graphique 10-17 ci-dessus ; nous allons compter uniquement sur la méthode Elbow pour nous indiquer la bonne valeur du paramètre *k* ! Pour cela, poursuivons avec notre exemple.

© Editions ENI - All rights reserved

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
fig, axs = plt.subplots(4, 3)
inertie_lst = []
k=1
for row in range(4):
   for col in range(3):
       print("k= ",k)
       model = KMeans(n_clusters=k)
       model.fit(data)
       inertie_lst.append(model.inertia_)
       centers = model.cluster_centers_
       axs[row,col].set_title(f"k = {str(k)}, inertie =
                              {model.inertia_}")
       axs[row,col].scatter(centers[:,0],
                            centers[:,1],c='red',s=100)
       axs[row,col].scatter(data.X, data.Y,c='#000000',s=1)
       for i in range(len(model.labels_)) :
           axs[row,col].text(data.X[i], data.Y[i],
                             model.labels_[i])
       k+=1

fig.set_figheight(20)
fig.set_figwidth(15)
```

Ce dernier code permet d'exécuter douze fois l'algorithme k-means en variant à chaque exécution la valeur du paramètre *k* en commençant par la valeur `k=1` jusqu'à la valeur `k=12`.

La première instruction permet de créer une grille de douze graphiques, quatre lignes et trois colonnes, où seront illustrés les résultats de chacune des douze exécutions de k-means. La liste `inertie_lst` permet de stocker le contenu de l'attribut `inertia_` associé à chaque modèle construit après chaque exécution. Les deux boucles avec les deux variables `row` et `col` permettent de placer les résultats de chaque exécution dans la case correspondante de la grille des graphiques. La boucle avec la variable `i` permet de mentionner sur chaque graphique le numéro de cluster de chaque point. Également, ce code permet d'afficher dans chaque graphique de chaque résultat les centres des clusters avec un grand point rouge. Cela va nous permettre de situer le centre de chaque groupement de données.

Les deux dernières instructions permettent tout simplement d'ajuster la taille des graphiques. L'exécution de ce code donne les résultats illustrés dans les quatre figures suivantes :

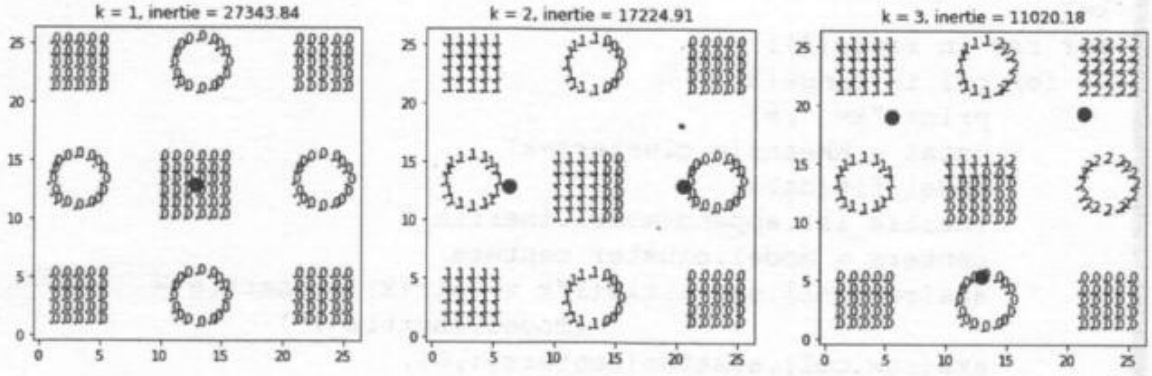


Figure 10-18 : clustering avec k-means et avec k=1, k=2 et k=3 sur les données exemple_simple_kmeans-3.csv

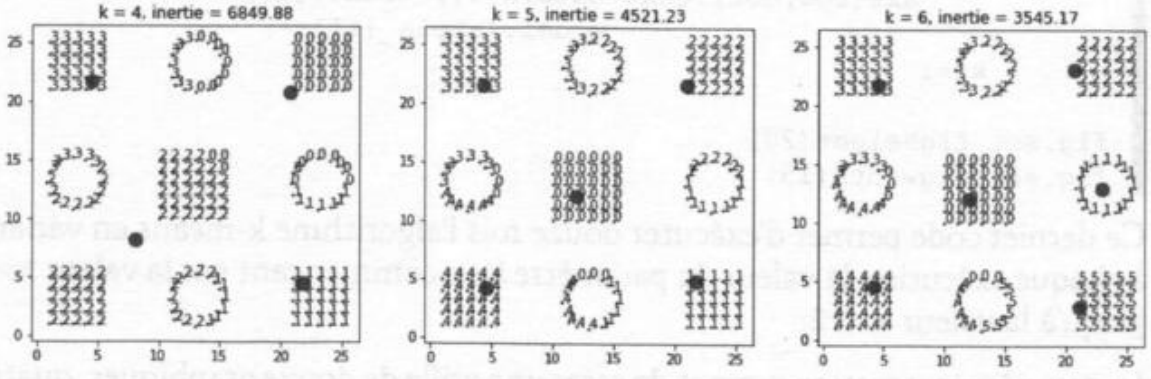


Figure 10-19 : clustering avec k-means et avec k=4, k=5 et k=6 sur les données exemple_simple_kmeans-3.csv

© Editions ENI - All rights reserved

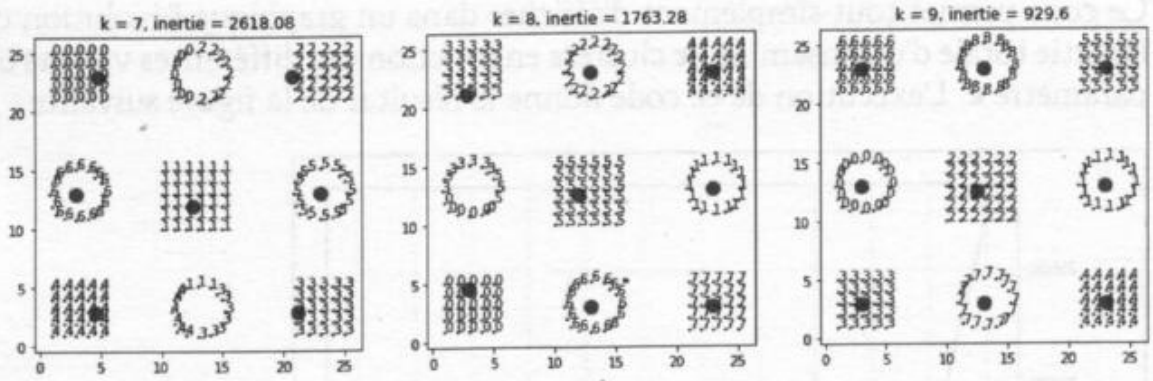


Figure 10-20 : clustering avec k-means et avec k=7, k=8 et k=9 sur les données exemple_simple_kmeans-3.csv

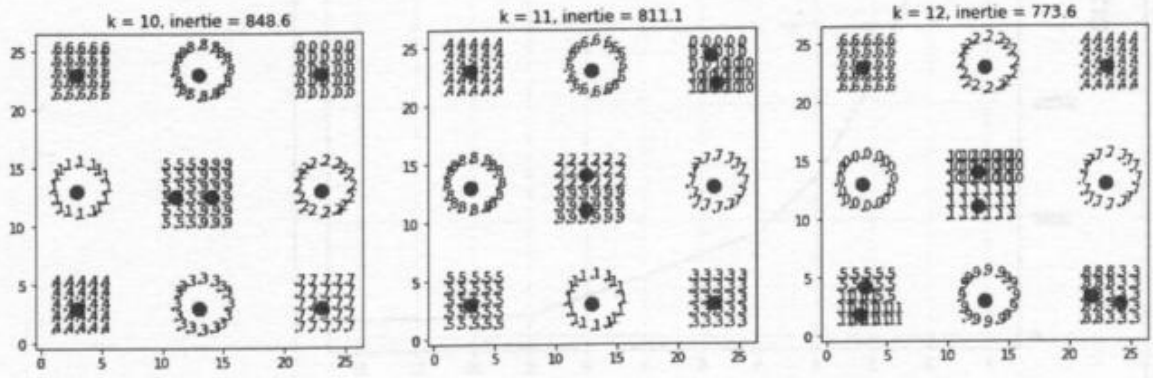


figure 10-21 : clustering avec k-means et avec k=10, k=11 et k=12 sur les données exemple_simple_kmeans-3.csv

Ces résultats montrent que plus la valeur du paramètre *k* augmente, c'est-à-dire plus il y a de clusters, plus la valeur de l'inertie totale est faible. Justement, grâce à la liste `inertie_lst` qui contient les inerties des douze exécutions de k-means, nous allons pouvoir tracer la courbe associée à la méthode Elbow.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
fig, ax = plt.subplots(1)
ax.set_xticks(np.arange(15))
ax.plot(range(1,13), inertie_lst)
ax.set_xlabel("k", fontsize=20)
ax.set_ylabel("Inertie", fontsize=20)
fig.set_figheight(8)
fig.set_figwidth(8)
plt.grid()
```

Ce code permet tout simplement d'afficher dans un graphique l'évolution de l'inertie totale d'un ensemble de clusters en fonction des différentes valeurs du paramètre *k*. L'exécution de ce code donne le résultat de la figure suivante :

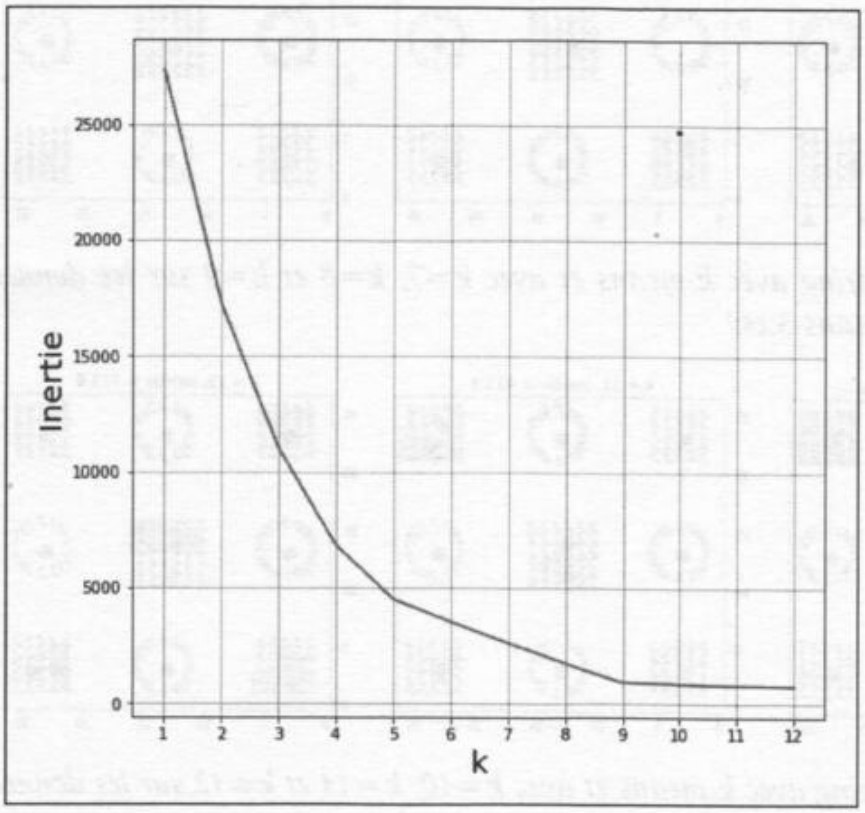


Figure 10-22 : courbe d'évolution de l'inertie totale en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-3.csv

Sur la courbe de la dernière figure, on voit que le chiffre 9 correspond à une valeur particulière. En effet, à partir de la valeur de k=9, la valeur de l'inertie totale est relativement stable. On peut clairement observer sur cette figure que la valeur 9 correspond à la dernière valeur qui a permis une diminution drastique de l'inertie.

En analysant cette courbe de manière plus approfondie, on remarque que l'inertie diminue au fur et à mesure que la valeur du paramètre k devient plus grande. En revanche, on constate que cette diminution de l'inertie est brutale avant la valeur 9 du paramètre *k* ; puis cette diminution devient plus lente après la valeur 9.

© Editions ENI - All rights reserved

Cette courbe peut être vue comme un bras en flexion dont l'épaule, qui n'est pas représentée dans le graphique, se situerait tout en haut à gauche, et la main, qui n'est pas non plus représentée dans le graphique, serait située tout en bas à droite. La valeur 9 serait le coude de ce bras imaginaire, d'où le nom de cette méthode !

Dans cet exemple, on voit que la valeur 9 correspond au coude du bras associé à cette courbe. Nous pouvons donc supposer que la valeur 9 du paramètre k est intéressante et que nos données seraient bel et bien organisées en neuf clusters. C'est ainsi que la méthode Elbow nous aide à déterminer le bon nombre de clusters.

Dans la suite de ce chapitre, nous allons voir que la méthode Elbow n'est pas toujours exploitable, car la courbe de l'évolution de l'inertie en fonction des valeurs du paramètre k n'est pas toujours facile à lire et à interpréter comme dans notre exemple. C'est pour cette raison que l'utilisation de la méthode Elbow est souvent combinée avec l'utilisation d'autres indicateurs tels que le coefficient de silhouette que nous allons découvrir ci-après.

6.2 Déterminer k avec le coefficient de silhouette

La méthode du coefficient de silhouette repose elle aussi sur l'exécution de k-means avec plusieurs valeurs du paramètre k, puis de réaliser des calculs sur les clusters construits à chaque exécution afin de déterminer le coefficient de silhouette de chaque partitionnement. Une fois calculé, ce coefficient de silhouette sera utilisé pour mesurer la qualité d'une partition.

Supposons k clusters associés à une exécution de k-means. Pour calculer le coefficient de silhouette sur l'ensemble de cette partition, il est nécessaire de calculer d'abord le coefficient de silhouette de chaque point x_i de chaque cluster.

Le coefficient de silhouette x_i d'un point x_i se calcule comme suit :

$$-1 \leq s(x_i) = \frac{b(x_i) - a(x_i)}{\max(a(x_i), b(x_i))} \leq 1 \quad (1)$$

Avec $a(x_i)$ et $b(x_i)$ définis comme suit :

- $a(x_i)$ est un coefficient de cohésion du point x_i avec les autres points de son propre cluster et qui se calcule comme suit :

$$a(x_i) = \frac{1}{|C^{x_i}| - 1} \sum_{j \in C^{x_i}, j \neq i} d(x_i, x_j)$$

Où C^{x_i} est le cluster d'appartenance du point x_i et donc $|C^{x_i}|$ représente le nombre d'éléments, ou le nombre de points, dans ce cluster.

Le facteur $a(x_i)$ correspond à la distance moyenne qui sépare le point x_i de tous les autres points de son cluster d'appartenance. De ce fait, $a(x_i)$ permet de comparer les distances intracluster. En règle générale, un clustering efficace est un clustering qui présente des facteurs $a(x_i)$ avec des valeurs faibles.

- $b(x_i)$ est le coefficient de séparation ou de divergence d'un point x_i avec le cluster voisin de son cluster d'appartenance. Il se calcule comme suit :

$$b(x_i) = min_{C \neq C^{x_i}} \frac{1}{|C|} \sum_{x_j \in C} d(x_i, x_j)$$

Pour calculer cette formule pour un point x_i, il est nécessaire de calculer les distances qui séparent ce point x_i avec tous les autres points de tous les clusters autres que le cluster d'appartenance de x_i. Ainsi, pour un cluster C autre que C^{x_i}, nous pouvons calculer la distance moyenne qui sépare le point x_i et les points de ce cluster C. C'est le cluster qui donne la plus petite valeur de cette distance moyenne qui sera considéré comme étant le cluster voisin du cluster C^{x_i}.

Le facteur $b(x_i)$ est aussi appelé la distance moyenne inter-clusters puisqu'il s'intéresse aux distances entre un point et les points des autres clusters.

© Editions ENI - All rights reserved

Dans la formule (1) ci-dessus, si la valeur de $s(x_i)$ est proche de 1, alors le point x_i est bien classé, et si la valeur de $s(x_i)$ est proche de -1, alors le point x_i est fort probablement mal classé. En revanche, si $s(x_i)$ est proche de 0, alors le point x_i peut être classé dans les deux clusters, celui où il est classé ou bien le cluster voisin le plus proche.

Avec les différents coefficients de silhouette $s(x_i)$, nous pouvons maintenant calculer le coefficient de silhouette moyen de toute une partition comme suit :

$$S = \frac{1}{k}\sum_{j=1}^{k}\frac{1}{|C^j|}\sum_{i\in C^j} s(x_i)$$

Le coefficient de silhouette S correspond à la moyenne des moyennes des coefficients $s(x_i)$. C'est-à-dire qu'une fois que la moyenne des $s(x_i)$ est calculée au sein de chaque cluster, nous obtiendrons k moyennes, S est la moyenne de toutes ces k moyennes.

La méthode du coefficient de silhouette en pratique

À travers un exemple, nous allons montrer comment calculer le coefficient de silhouette pour une partition et comment cette mesure peut nous aider dans notre prise de décision quant à la sélection de la valeur du paramètre k de l'algorithme k-means. Pour la réalisation de ce nouvel exemple de code, réutilisons le même jeu de données que dans l'exemple précédent.

Remarque

Le code de cet exemple est quasiment le même que le code de l'exemple précédent. De ce fait, les explications ci-dessous sont concentrées uniquement sur les différences entre ces deux codes.

- Créez un nouveau notebook, puis saisissez et exécutez les mêmes instructions d'import de modules que dans l'exemple précédent en ajoutant l'instruction d'import ci-dessous ; vous pouvez utiliser le notebook *k-means-04.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
from sklearn.metrics import silhouette_score
```

La fonction `silhouette_score` nous permet de calculer le coefficient de silhouette associé à une partition de données.

- Dans une nouvelle cellule Jupyter, utilisez le même code que dans l'exemple précédent afin de lire les données du fichier *exemple_simple_kmeans-3.csv* et d'exécuter l'algorithme k-means avec des valeurs du paramètre `k=[2,3,4,...,13]` ainsi que pour calculer les coefficients de silhouette associés à chaque exécution comme dans le code suivant :

```
data = pd.read_csv('../Data/exemple_simple_kmeans-3.csv',
                   sep=',')
plt.figure(1, figsize=(5, 5))
plt.scatter(data.X, data.Y,s=60)

fig, axs = plt.subplots(4, 3)
inertie_lst = []
silhouette_lst = []

k=2 #Ici, la première valeur du paramètre k est égale à 2.

for row in range(4):
    for col in range(3):
        print("k= ",k)
        model = KMeans(n_clusters=k)
        model.fit(data)
        inertie_lst.append(model.inertia_)

        #Calcul du coefficient de silhouette
        silhouette_lst.append(silhouette_score(data,
                                               model.labels_))

        centers = model.cluster_centers_
        axs[row,col].set_title(f"k = {str(k)}, inertie =
                               {round(model.inertia_,2)}")
        axs[row,col].scatter(centers[:,0],
                             centers[:,1],c='red',s=100)
        axs[row,col].scatter(data.X, data.Y,c='#000000',s=1)
        for i in range(len(model.labels_)) :
            axs[row,col].text(data.X[i], data.Y[i],
                                         model.labels_[i])
        k+=1

fig.set_figheight(20)
fig.set_figwidth(15)
```

© Editions ENI - All rights reserved

Les deux seules différences entre ce nouveau code et celui de l'exemple précédent sont les suivantes :

- Le paramètre *k* varie entre la valeur 2 et la valeur 13. La première valeur est égale à 2 car le coefficient de silhouette n'a d'intérêt que si le nombre de clusters est strictement supérieur à 1. La dernière valeur du paramètre *k* est naturellement égale à 13, puisqu'avec ce code nous réalisons douze exécutions de k-means afin d'avoir douze graphiques.
- L'utilisation de la fonction `silhouette_score` permet de calculer le coefficient de silhouette de chaque exécution de k-means avec une valeur différente du paramètre *k*. Le coefficient de silhouette associé à chaque exécution de k-means est ajouté à la liste `silhouette_lst`.

À la fin de l'exécution de la dernière cellule Jupyter, nous avons stocké dans la liste `silhouette_lst` toutes les valeurs du coefficient de silhouette.

▶ Dans une nouvelle cellule Jupyter, utilisez le code suivant afin d'afficher le graphique de l'évolution du coefficient de silhouette en fonction des valeurs du paramètre *k*.

```
fig, ax = plt.subplots(1)
ax.set_xticks(np.arange(15))
ax.plot(range(2,14), silhouette_lst)
ax.set_xlabel("k", fontsize=20)
ax.set_ylabel("Silhouette", fontsize=20)
fig.set_figheight(8)
fig.set_figwidth(8)
plt.grid()
```

L'exécution de ce code donne le graphique ci-dessous :

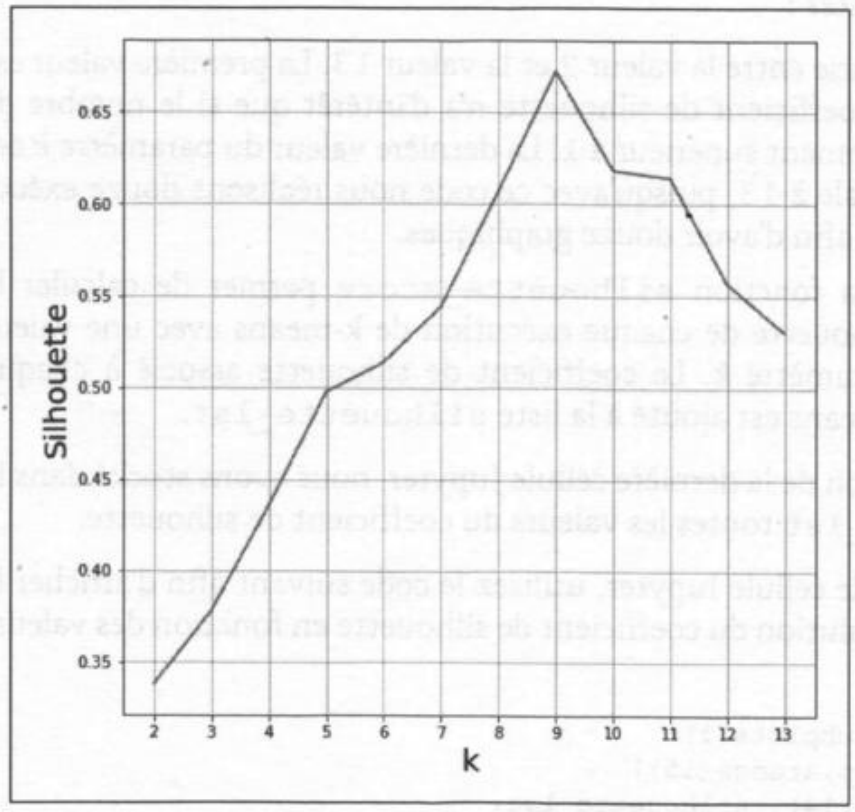


Figure 10-23 : courbe d'évolution du coefficient de silhouette en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-3.csv

Ce graphique montre que la valeur du coefficient de silhouette moyen atteint une valeur maximale avec la valeur du paramètre k=9.

Dans ces deux derniers exemples, nous avons choisi un jeu de données qui permet d'illustrer l'utilisation des deux métriques de performance d'un clustering basées sur les méthodes Elbow et du coefficient de silhouette. Bien que ces deux métriques nous permettent de mesurer la qualité d'un clustering de manière quantitative, elles ne peuvent cependant pas être utilisées pour automatiser complètement le processus de prise de décision pour la détection de la meilleure valeur du paramètre *k*. À l'état actuel des choses, la détection du bon nombre de clusters à considérer requiert l'intervention d'experts humains ! Ces deux métriques sont plutôt un support qui nous conforte dans notre décision quant à la valeur du paramètre *k*.

© Editions ENI - All rights reserved

Il faut noter également que les deux métriques présentées ci-dessus ne sont pas spécifiques à l'algorithme k-means et peuvent être utilisées pour mesurer la qualité d'un clustering construit avec n'importe quel autre algorithme.

7. Les limites de k-means

Dans cette section, nous allons montrer un exemple où l'algorithme k-means ne réussira pas à répartir les données dans des clusters de façon efficace. Cela va nous permettre de mettre en lumière les faiblesses de cet algorithme. Également, ce nouvel exemple illustrera un cas où les deux métriques Elbow et le coefficient de silhouette ne nous seront pas d'une grande aide !

En effet, comme nous allons le voir, les performances de l'algorithme k-means dépendent directement de l'organisation des données.

- Pour la réalisation de ce nouvel exemple, réutilisez votre notebook Jupyter créé lors de l'exemple précédent ou réutilisez le notebook *k-means-04.ipynb*. Dans les deux cas, la seule modification nécessaire est de remplacer le nom du fichier *exemple_simple_kmeans-3.csv* par *exemple_simple_kmeans-4.csv* à partir duquel vous allez lire et utiliser un nouveau jeu de données. Une fois cette modification effectuée, exécutez toutes les cellules de ce notebook afin de pouvoir visualiser les résultats. Ces derniers sont présentés ci-après.

La visualisation de ce nouveau jeu de données est illustrée dans la figure suivante :

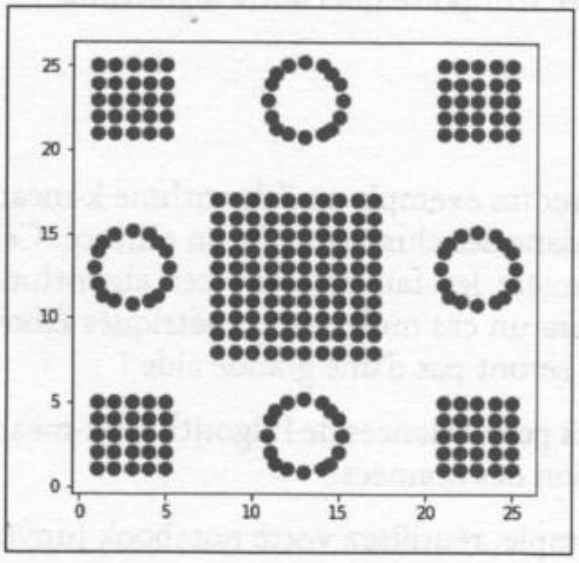


Figure 10-24 : représentation graphique des données du fichier exemple_simple_k-means-4.csv

Ce nouveau jeu de données ressemble beaucoup au jeu de données utilisé précédemment (voir figure 10-17), sauf que cette fois le groupe de données central contient un peu plus de points et il se rapproche grandement des groupes de données sur les bords. Nous allons voir que malgré cette différence de données relativement minime, elle aura un impact majeur sur le fonctionnement de l'algorithme k-means !

La visualisation du graphique de la figure 10-24 montre nettement que les données sont organisées dans neuf groupes. Cependant, avec la valeur k=9, k-means n'a pas réussi à positionner correctement tous les points comme le montre la figure 10-25. En effet, ces résultats montrent que les points du lot central ont été répartis dans les trois clusters représentés avec les chiffres 4, 7 et 8.

On note également que le centre du cluster représenté avec le chiffre 4 se situe en dehors du cercle formé par les points du lot au milieu en bas. Aussi, le centre du cluster représenté avec le chiffre 7 est quasiment situé sur le bord du cercle formé par les points du lot au milieu à droite.

© Editions ENI - All rights reserved

On peut expliquer ces résultats en observant que les points situés tout en bas du lot central sont plus proches des points en haut du lot au milieu en bas que des points situés tout en haut de ce même lot central ! Il en est de même pour les points situés tout à gauche du lot central qui sont plus proches des points à droite du lot au milieu à gauche que des points à droite de ce lot central !

Bien sûr, force est de constater que ces résultats sont de moindre qualité que ceux de la figure 10-20 avec k=9.

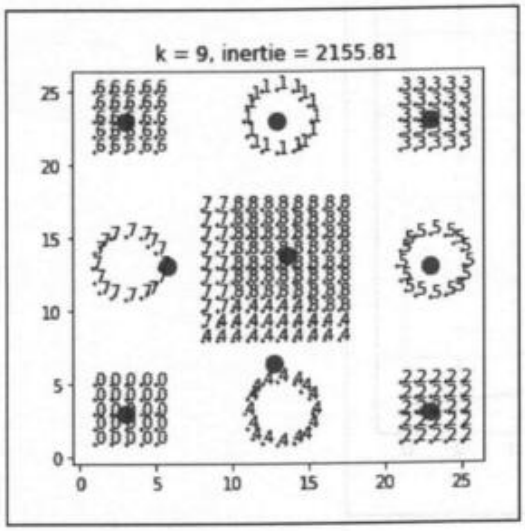


Figure 10-25 : clustering avec k-means et avec k=9 sur les données exemple_simple_kmeans-4.csv

La figure suivante trace la courbe Elbow correspondant à l'évolution de l'inertie totale en fonction de la valeur du paramètre *k* :

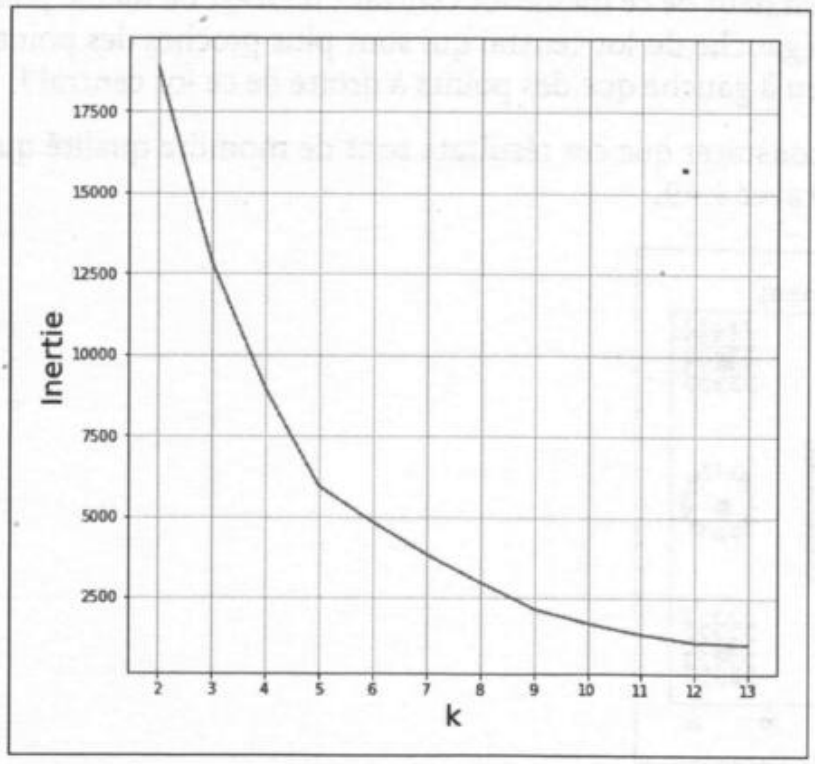


Figure 10-26 : courbe d'évolution de l'inertie en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-4.csv

Comparativement à la courbe de la figure 10-22, cette dernière nous laisse incertains quant au choix de la valeur du paramètre Aller à la page précédente, car, pour rappel, le fait que nous sachions que `k=9` est la valeur optimale n'est dû qu'au fait que nous pouvons visualiser ces données. Ici, dans cette nouvelle courbe, nous aurions du mal à choisir entre la valeur `9` et la valeur `12` !

La figure suivante montre la courbe de l'évolution du coefficient de silhouette en fonction des différentes valeurs de *k*.

© Editions ENI - All rights reserved

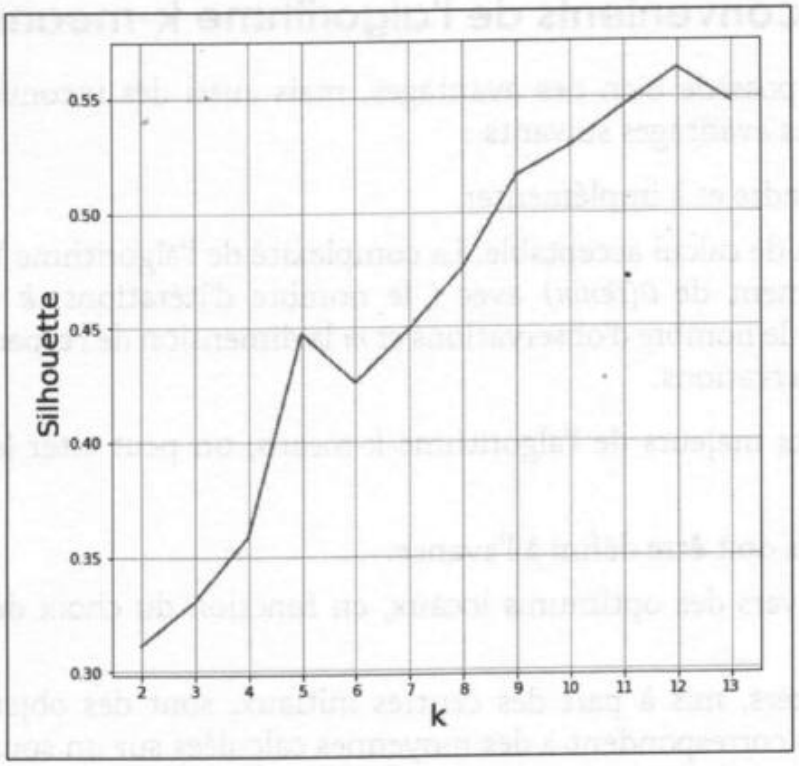


Figure 10-27 : courbe d'évolution du coefficient de silhouette en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-4.csv

La courbe de l'évolution du coefficient de silhouette décrit un pic pour une valeur de `k=12` !

Cet exemple prouve que si notre analyse pour trouver la meilleure valeur du paramètre *k* n'était basée que sur les deux critères de performance Elbow et le coefficient de silhouette, alors nous n'aurions pas forcément considéré que `k=9` est la meilleure valeur !

En synthèse, l'utilisation de méthodes de mesure de la qualité d'un clustering, telles que les méthodes Elbow et du coefficient de silhouette, ne doit pas éliminer complètement l'analyse par un expert du contenu des différents clusters ! En effet, seule une analyse métier peut confirmer la véracité d'un clustering, et toutes les méthodes mathématiques de mesure de qualité d'un clustering ne devraient être utilisées que comme un support et une aide à la décision.

8. Avantages et inconvénients de l'algorithme k-means

L'algorithme k-means possède bien des avantages, mais aussi des inconvénients. On peut citer les avantages suivants :

- Il est facile à comprendre et à implémenter.
- Il implique un temps de calcul acceptable. La complexité de l'algorithme k-means est effectivement de $O(iknm)$ avec i le nombre d'itérations, k le nombre de clusters, n le nombre d'observations et m la dimension de l'espace de définition des observations.

Parmi les inconvénients majeurs de l'algorithme k-means, on peut citer les points suivants :

- Le nombre de clusters doit être défini à l'avance.
- Il converge souvent vers des optimums locaux, en fonction du choix des centres initiaux.
- Les centres des clusters, mis à part des centres initiaux, sont des objets inexistants puisqu'ils correspondent à des moyennes calculées sur un sous-ensemble d'observations à chaque itération.
- Une forte influence des valeurs aberrantes sur les résultats.
- Il donne des résultats médiocres pour les données qui ne sont pas linéairement séparables.
- Il n'est pas adapté aux données non numériques.

© Editions ENI - All rights reserved

9. Quelques versions de l'algorithme k-means

Pour pallier les inconvénients de l'algorithme k-means dans sa version classique abordée dans ce chapitre, plusieurs versions ont été développées. Sans entrer dans les détails, on peut citer parmi ces versions :

- L'algorithme k-medoids : cet algorithme suit le même schéma général que l'algorithme k-means, sauf que k-medoids choisit les centres des clusters parmi les observations déjà présentes. Par rapport à k-means, il présente les deux avantages majeurs suivants :
  - Il est beaucoup moins sensible aux valeurs aberrantes.
  - Les distances entre les observations peuvent être calculées au début de l'algorithme puisque les centres des clusters sont représentés par des observations déjà existantes. C'est un avantage dans la mesure où le nombre d'itérations nécessaires est supérieur au nombre d'observations !
- L'algorithme k-modes : cet algorithme est une adaptation de l'algorithme k-means pour traiter les données de type non numériques. Le centre d'un cluster est calculé en fonction des fréquences des modalités majoritaires présentes dans ce cluster.
- L'algorithme CLARA : cet algorithme est adapté aux données de très grande taille. Il procède par application d'une version de k-means appelée PAM (*Partitioning Around Medoids*) sur plusieurs échantillons des observations initiales.

10. Conclusion

Après avoir présenté le fonctionnement de l'algorithme k-means via des illustrations graphiques, ce chapitre a présenté quelques exemples de code d'utilisation de k-means avec Scikit-learn en portant une attention particulière à la notion de mesure de qualité d'un clustering avec la présentation des deux méthodes Elbow et du coefficient de silhouette. Également, ce chapitre a abordé les limites de l'algorithme k-means via un exemple et évoqué ses avantages et ses inconvénients. Nous avons conclu avec une brève présentation de quelques versions de l'algorithme k-means.